
Tomo

Read, write, and connect.

Background

Tomo serves a large global community of fanfiction readers and writers who are highly active but lack a centralized modern platform. Most rely on a small number of existing platforms that are often outdated, poorly optimized for mobile, or no longer actively maintained. As a result, the community feels “homeless” despite its size and engagement. These users prioritize creative freedom and seek a platform designed specifically for fanfiction that supports long-form reading, detailed tagging systems, and active community discussions.

Problem Statement

Although the fanfiction community continues to grow, the digital platforms supporting it have not kept pace. Users face several key challenges including limited platform choices, outdated or abandoned apps, and poor mobile experiences due to a lack of app-first design. Features that modern users expect such as offline reading, real-time interaction, and seamless syncing are often missing. In addition, the social aspect of fandom is fragmented with discussions happening on separate platforms, making it harder for readers and writers to connect.

Solution

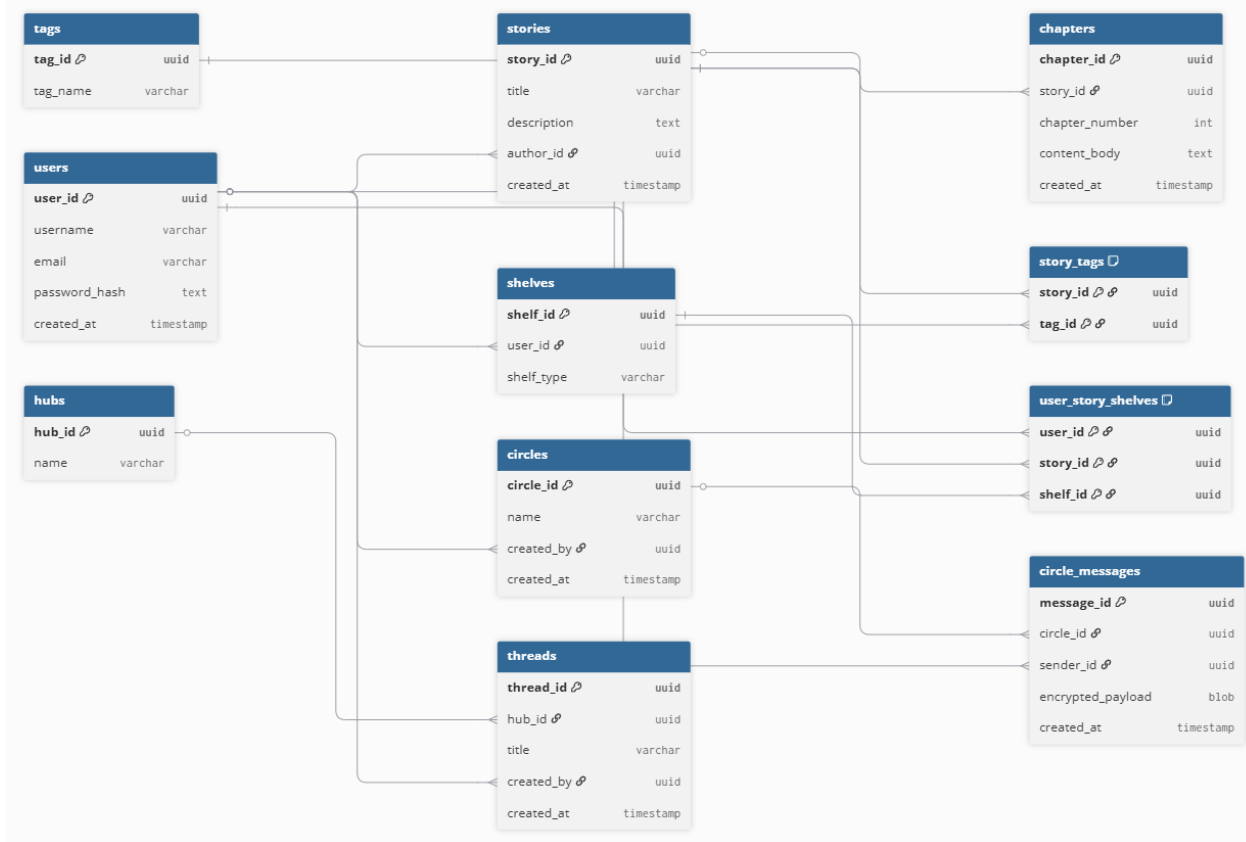
Tomo addresses these gaps by providing a modern all-in-one mobile application where users can read, write, and interact within a single high-performance ecosystem tailored to the needs of the fanfiction community.

Data Design

The Data Design ensures that Tomo can handle complex many-to-many relationships (like a story having multiple tags) and the offline "Vault" capability.

Data Structures

- **B-Trees:** Used for high-speed indexing in the PostgreSQL database to ensure that "Exclude/Include" tag filtering remains fast as the archive grows.
- **Client-side queue:** Manages Auto-Cache operations for sequential chapter preloading.
- **Hash Maps:** Used within the **Intelligence Service** to store and weigh "Library DNA" preferences for instant lookups.
- **Encrypted message payloads:** Stores ciphertext for private 'Circles' messages, ensuring end-to-end encryption.



ii. Database Schema

The schema follows a Relational SQL approach to manage the deep interconnectedness of users, stories, and social features while maintaining scalability and data integrity.

Core Relationships:

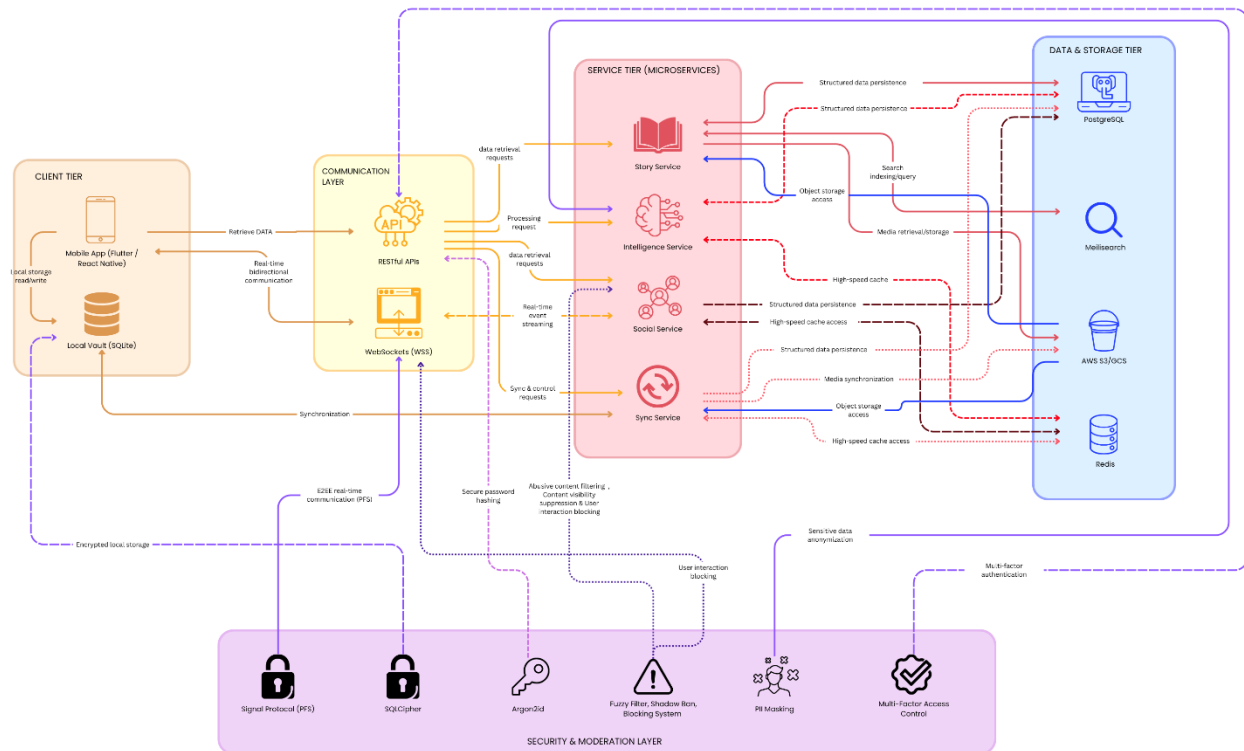
- **Users & Stories (Author)(One-to-Many):** One user can author multiple stories.
- **Users & Stories (Many-to-Many):** Implemented through *shelves* and the *user_story_shelves* junction table (e.g., Favorites, To-Read).
- **Stories & Tags (Many-to-Many):** Implemented by the *story_tags* junction table, enabling flexible Include/Exclude filtering.
- **Hubs & Threads (One-to-Many):** Each fandom hub contains multiple discussion threads.
- **Stories & Chapters (One-to-Many):** Each story consists of multiple chapters ordered by sequence.
- **Users & Shelves (One-to-Many):** Each user can create and manage multiple shelves.

iii. Data Dictionary (Key Entities)

Entity	Attribute	Data Type	Description
User	user_id	UUID	Primary Key; unique identifier for each account.
Story	story_id	UUID	Primary Key; uniquely identifies a story and links to its author.
Chapter	chapter_id	UUID	Primary Key; uniquely identifies each chapter.
	content_body	LongText	The actual story text (optimized for lazy-loading).
Shelf	shelf_type	ENUM	Categorizes stories (Reading, Favorites, To-Read, DNF).
Circle Message	encrypted_payload	Blob	Encrypted message content for private group communication (End-to-End Encryption).

Architectural Design

Tomo utilizes a Cloud-Native Hybrid Architecture, prioritizing an "Offline-First" logic where the local device acts as a primary workspace.



System Architecture (Ideal Design)

Client Tier

The Client Tier is built using a cross-platform framework such as Flutter or React Native, ensuring consistent performance across both iOS and Android devices. It includes the **Local Vault (SQLite)**, which enables the application's Offline-First functionality by storing story data directly on the user's device.

Communication Layer

- **RESTful APIs:**
These handle standard request-response interactions, such as retrieving user profiles, story metadata, and library indexing data.
- **WebSockets (WSS):**
These maintain a persistent, two-way connection that supports real-time features, including Buddy Reading and Circles chat.

Service Tier (Microservices)

- **Story Service:**
Responsible for efficient Markdown rendering and handling complex archive search operations.
- **Social Service:**
Manages community features such as forums, private circles, and automated moderation.
- **Intelligence Service:**
Processes “Library DNA” to generate the personalized Active Pulse discovery feed.
- **Sync Service:**
Ensures data consistency by managing synchronization, version control, and conflict resolution between the Local Vault and the cloud data layer.

Data & Storage Tier

- **PostgreSQL:**
Serves as the primary relational database, managing structured relationships such as users, stories, and tags.
- **Meilisearch:**
Functions as the dedicated search engine for the archive, enabling fast and typo-tolerant queries across large datasets.
- **Redis:**
Used for session management and high-speed caching, particularly for frequently accessed data like the Active Pulse feed.
- **AWS S3:**
Provides scalable cloud storage for large media assets, including high-resolution cover images.

II. Security & Moderation Layer

Data Privacy and Encryption

- **Signal Protocol with Perfect Forward Secrecy (PFS):**
Ensures End-to-End Encryption (E2EE) for private Circles. PFS guarantees that even if a session key is compromised, past communications remain secure.
- **SQLCipher:**
Encrypts data stored in the Local Vault, protecting downloaded content and private drafts in case of device loss or theft.
- **Argon2id:**
A highly secure password hashing algorithm used to protect user credentials against brute-force and GPU-based attacks.

Behavioral Governance Suite

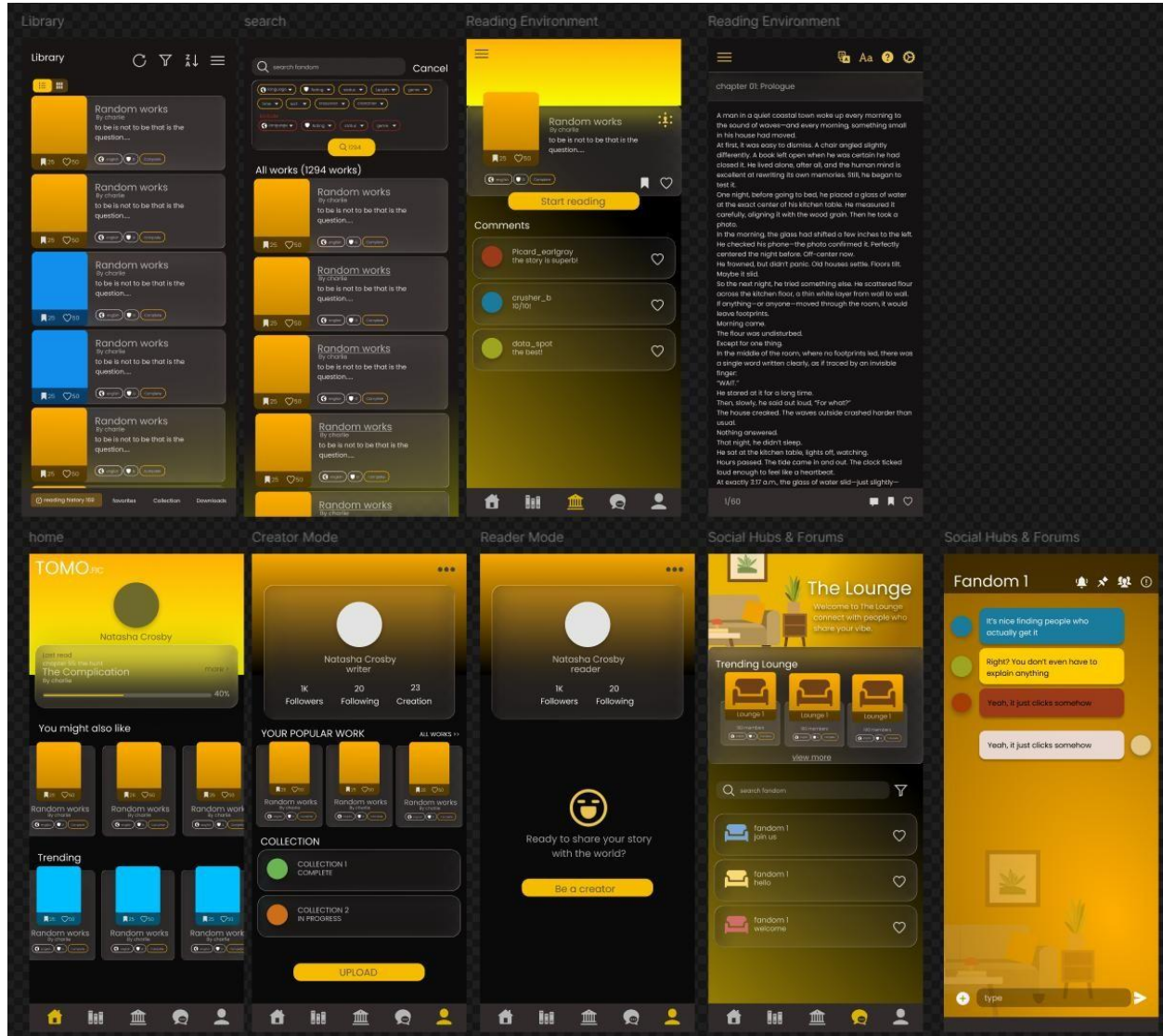
- **Fuzzy Filter:**
An advanced content filtering system that detects harmful or abusive language, even when users attempt to bypass filters through altered spelling or symbols.
- **Shadow Ban:**
A moderation approach that limits the visibility of disruptive users' content without their awareness, reducing conflict escalation.
- **Blocking:**
Enforces strict separation between users across the system, preventing any form of interaction, preventing any form of interaction.

Access Control and Privacy Measures

- **PII Masking:**
A privacy safeguard that removes personally identifiable information (such as names and email addresses) before data is processed by the Intelligence Service.
 - **Multi-Factor Access Control:**
Enhances account security through methods such as email verification. These measures help prevent unauthorized access and reduce the risk of repeated abuse by banned users.
-

Interface Design

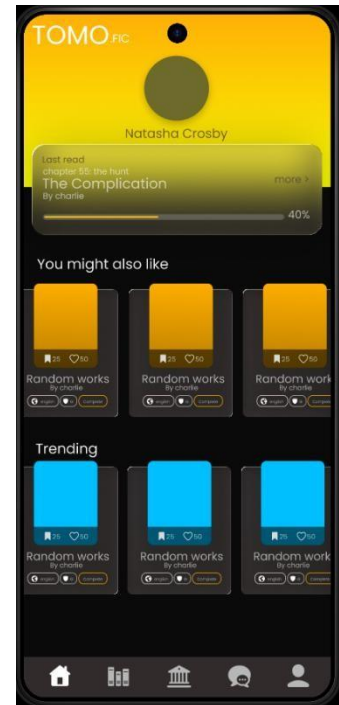
The Tomo interface is designed with a User-Centric approach, focusing on accessibility, minimalist aesthetics to reduce "reader fatigue," and seamless transitions between reading and community engagement.



Home Page

The landing page serves as the user's command center, designed to minimize the time between opening the app and resuming a story.

- **User Header:** Located at the upper middle, displaying the user's profile picture and display name for a personalized greeting.
- **Recent Activity (Last Read):** Positioned prominently below the header. It displays a "Continue Reading" card featuring the story title, chapter number, and a visual progress bar (percentage) to allow one-tap access to the exact point the user left off.
- **Personalized Recommendations:** A horizontal feed generated by the Intelligence Service based on recent searches and "Library DNA."
- **Trending Fandom Hubs:** Located at the bottom to encourage exploration beyond the user's usual interests by showing what the broader community is currently reading.
- **Main Navigation:** A 5-button bottom navigation bar: **Home, Library, Archive (Search), Lounge, and Profile/Creator Mode.**



Library (Personal Collection)

The Library is the user's private repository, optimized for both online and offline use.

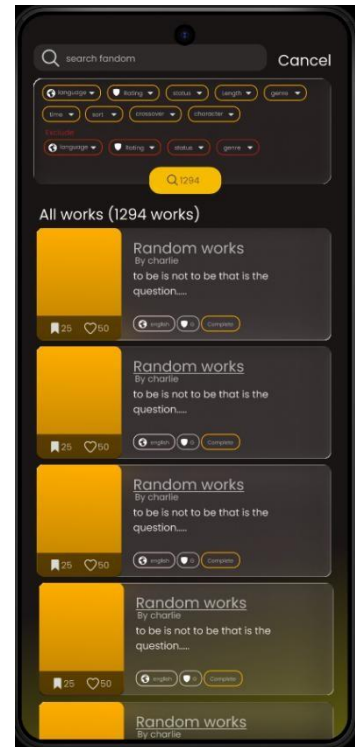
- **Categorization:** Divided into four clear tabs: **History** (Recents), **Favorites**, **Collections** (User-created folders), and **The Vault** (Downloaded works available 24/7 without internet).
- **Management Tools:** The upper corner features utility icons for **Refresh, Filter, Alphabetical/Chronological Sorting, and Library Settings.**



Search & Archive

A powerful tool designed for the specific needs of fanfiction discovery.

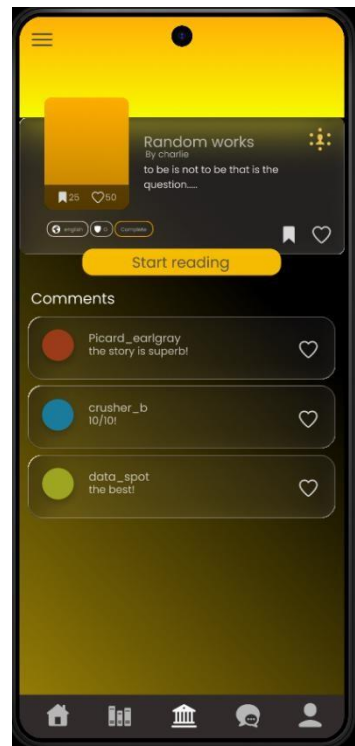
- **Omni-Search Bar:** Accepts keywords, specific character names, or fandom titles.
- **Complex Filtering:** A dedicated filter suite allows users to **Include** specific tropes/characters and **Exclude** unwanted content.
- **Search Results:** Displays the total count of available works found and list-view cards with metadata (word count, rating, and status).



Reading Environment: Cover Page

Before entering the text, users are presented with a rich metadata screen:

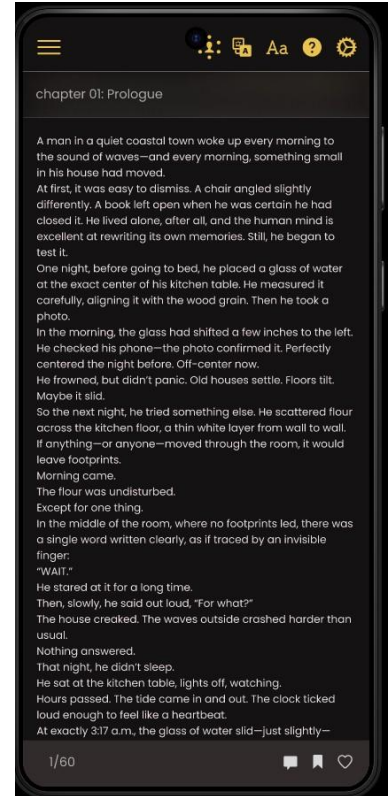
- **Metadata:** Displays cover art, title, author, full description, and story tags.
- **Social Metrics:** Shows the total number of Likes and Bookmarks.
- **Buddy Reading Indicator:** A status icon showing if the story is currently being shared for a real-time "Sync Reading" session with a friend.
- **Engagement:** An "Action Bar" containing the **Start Reading** button and a scrollable community comment section where users can read and "heart" reviews.



Reading Environment: Interactive Content

Designed for maximum focus and high customization.

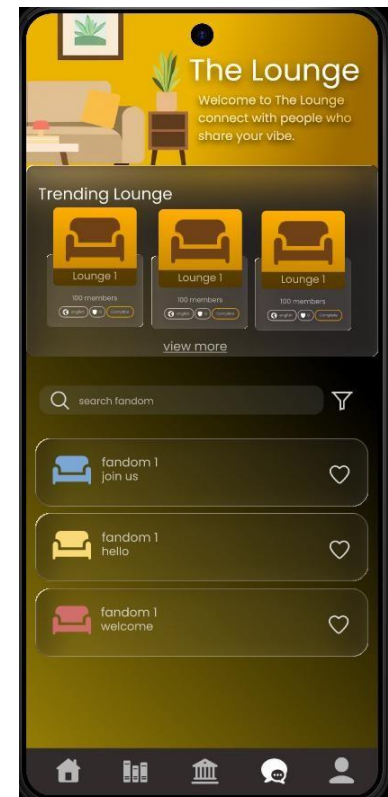
- **Minimalist Reader:** A clean text interface.
- **Navigation Slide-out:** A floating menu on the upper-left corner allows for rapid chapter selection.
- **Social & Accessibility Hub:** The upper-right menu features:
 - **Connect:** Real-time reading session invitation.
 - **Tools:** Translate and Help/Navigation guide.
 - **Preferences:** Deep customization for color themes (Sepia, OLED Black), font styles, text size, and **Dyslexic Mode** for better readability.
- **Footer Actions:** Displays current page number and floating buttons for **Comment, Bookmark, and Like**.



The Lounge & Forum

The community-driven space for fandom discussions.

- **Lounge Discovery:** Shows "Trending Social Lounges" and "Recommended for You" circles to help users find their specific community.
- **Navigation:** Includes a dedicated search bar and filter to locate specific fandom circles.



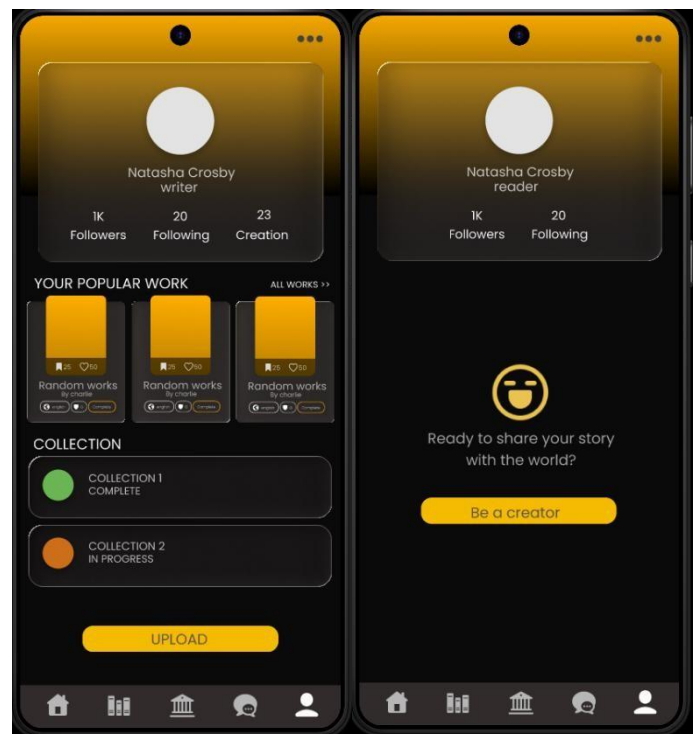
- **Lounge Forum (Group Chat):** A real-time discussion space featuring:
 - **Smart Notifications:** A bell icon to toggle alerts or mute the lounge.
 - **Pinboard:** A dedicated area to view important pinned messages.
 - **Member Directory:** View active users and identify **Moderators**.
 - **Safety Tools:** A prominent **Report Button** to flag foul behavior directly to the Moderation Layer.



Account & Dual-Mode Workspace

A dynamic profile that adapts based on the user's role.

- **Reader Mode:** A clean social profile displaying the user's bio, following list, and follower count.
- **Creator Mode Request:** Users must submit a request to unlock creator tools, ensuring a high standard of content within the app.
- **Creator Dashboard:** Once unlocked, the profile expands to show:
 - **Creation Metrics:** Total works published and cumulative statistics.
 - **Portfolio:** Showcases the user's most popular works and curated collections.

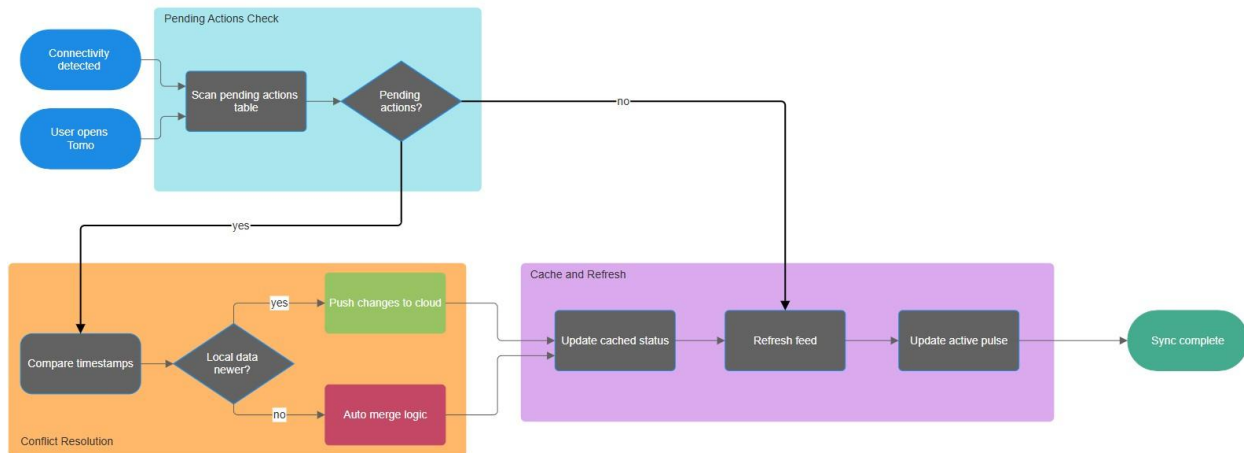


Procedural Design

The procedural design defines the logic flows that make Tomo unique, specifically focusing on data integrity and user interaction.

Flowchart: Smart-Sync Procedure

This flow governs how the app transitions from "The Vault" back to an active internet connection without losing data.



Pseudocode: Library DNA Algorithm

```
ALGORITHM GenerateActivePulseFeed(UserID)
  RecentFics = FetchLastRead(UserID, limit=10)
  VibeProfile = CalculateTagWeights(RecentFics)

  // Query Archive for matches excluding already read stories
  Recommendations = Archive.Search(
    Include = VibeProfile.Top(3),
    Exclude = User.AlreadyReadList
  )

  // Diversity Injection (Palette Cleansers)
  IF VibeProfile.Intensity > 0.8:
    Recommendations.Append(GetRandomGenre("Fluff"))

  RETURN Recommendations
END ALGORITHM
```